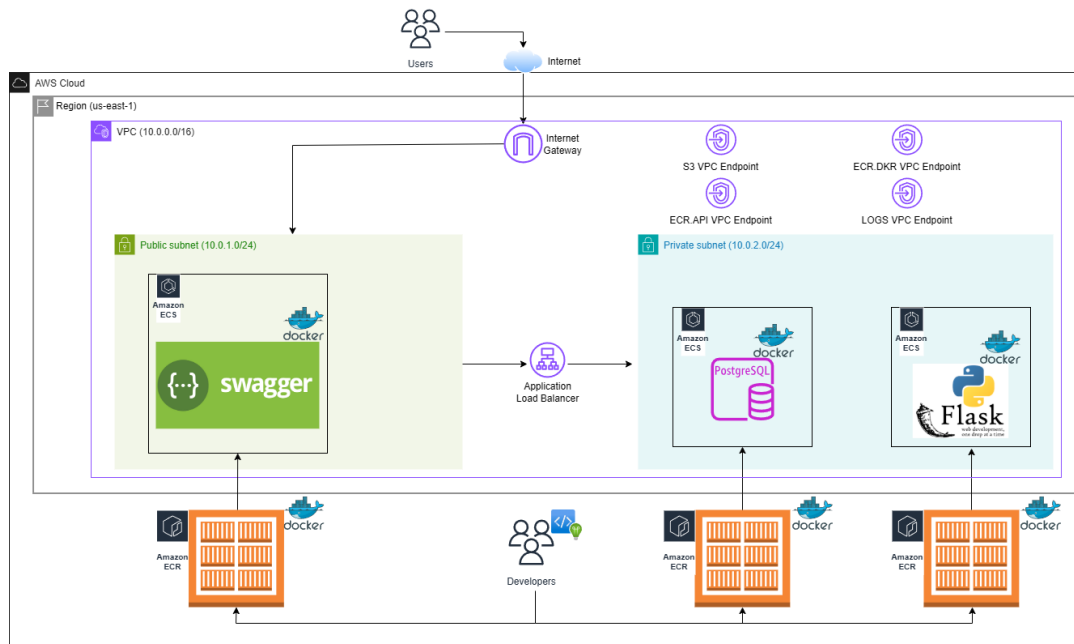




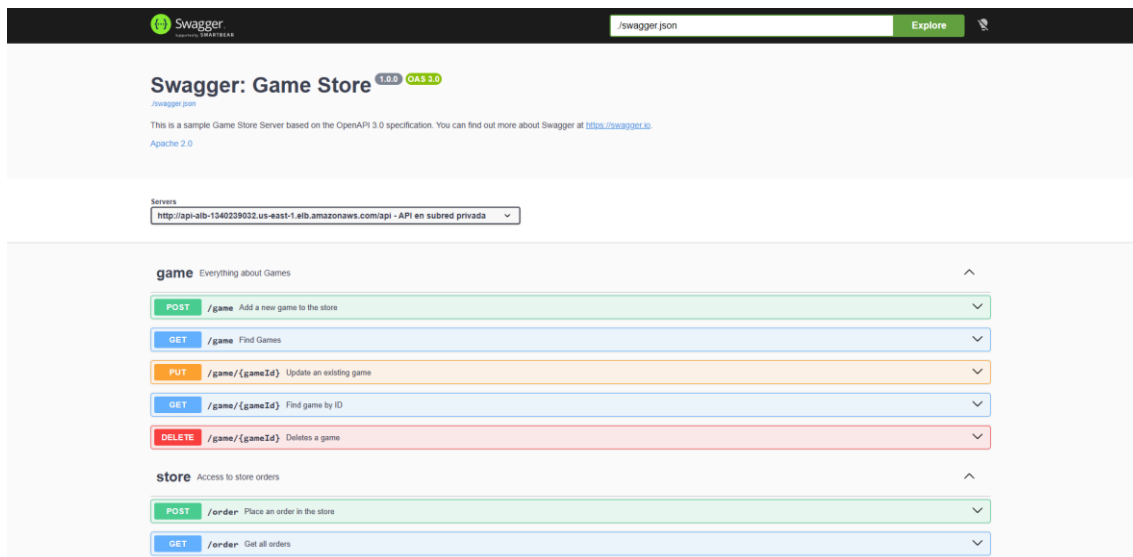
La primera arquitectura está desplegada en una misma región, en este caso, la de Virginia del Norte conocida como us-east-1. Dentro de esta región se crea una nube virtual (VPC) donde se albergan el resto de los recursos del despliegue.

El internet gateway permite a los usuarios de Internet acceder a los recursos en este caso disponibles en la subred pública. En esta subred pública se encuentra el servicio Swagger que se despliega como un contenedor de Docker, subiendo primero la imagen Docker a un repositorio ECR, luego creando una tarea que define la lógica de este contenedor y posteriormente se crea un servicio que contiene la configuración de red, seguridad o escalado entre otras propiedades.

En lo relativo a la subred privada, la lógica que se sigue es análoga con la peculiaridad de que al ser un segmento de red aislado del resto de Internet no existe manera de contactar con el servicio que contiene las imágenes de Docker para desplegar el contenedor. Este problema se ha resuelto usando VPC endpoints que permiten la comunicación entre algunos recursos AWS aun estando estos en zonas privadas. Se ha optado por esta opción de los VPC endpoints frente a la opción clásica para estos casos que es el NAT Gateway dado su reducido coste frente a esta última.

Merece la pena destacar que la imagen de PostgreSQL se obtiene directamente de Docker Hub no siendo necesaria ninguna configuración adicional al contrario que en los casos del servicio REST y el Swagger. Por último, para que Swagger y la API se puedan comunicar se ha optado por crear un balanceador de carga que permite esta conexión entre recursos públicos y privados dentro de una misma VPC.

En cuanto a lo que representa esta arquitectura, se está desplegando una API que define una serie de operaciones para una tienda virtual de juegos junto con una página Swagger que facilita y documenta el uso de esta API. Esto, sumado a una base de datos PostgreSQL que permite hacer las operaciones persistentes y almacena toda la información relacionada con la tienda, como pueden ser, los juegos y sus categorías, los usuarios o los pedidos.



En lo que se refiere al diseño del API, a continuación, se muestran sus métodos de forma tabular siendo las celdas verdes las operaciones disponibles:

	GET	POST	PUT	DELETE
/game				
/game/{gameId}				
/order				
/order/{orderId}				
/user				
/user/{username}				
/tag				
/tag/{tagId}				
/category				
/category/{categoryId}				

Y ahora se muestran los esquemas de los distintos objetos:

Order:

Nombre del campo	Tipo del campo
id	integer
games	[Game{...}, ...]
user	User{...}
status	string
address	string

Category:

Nombre del campo	Tipo del campo
name	string

User:

Nombre del campo	Tipo del campo
id	integer
username	string
firstName	string
lastName	string
email	string
password	string
phone	string

Tag:

Nombre del campo	Tipo del campo
name	string

Game:

Nombre del campo	Tipo del campo
id	integer
name	string
availableQuantity	integer
category	Category{...}
photoUrl	string
tags	[Tag{...}, ...]

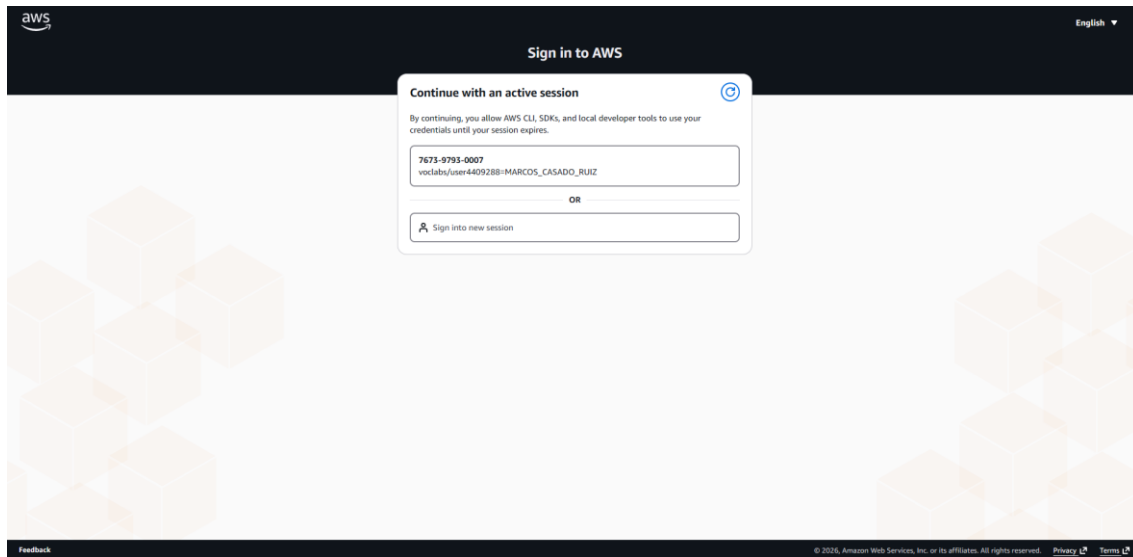
1.1 Pasos previos

Antes de proceder con el despliegue, es necesario remarcar los requisitos que han de cumplirse para realizar este, así como subrayar que no existe una única forma de realizar todo el proceso.

En cuanto a los requisitos previos, será necesario tener instalado en la máquina en la que se vaya a realizar este proceso tanto AWSCLIv2, como Python, por ser el lenguaje en el que está escrito el script, como Docker. También es imprescindible estar autenticado en AWS para tener los permisos necesarios y que así las llamadas a la API de Amazon no fallen. Esto último se puede conseguir de tres formas principalmente:

- 1) Mediante el mandato interactivo de credenciales AWS (*aws configure*) donde se solicitará al usuario ID de clave secreta (AWS Access Key ID), la clave secreta de acceso (AWS Secret Access Key), el token de la sesión (AWS Session Token), el nombre de la región por defecto (Default region name) que suele ser *'us-east-1'* y el formato de salida por defecto (Default output format) que suele dejarse en blanco lo que indica UTF-8.

- 2) A través del portal web de credenciales de AWS que se muestra a continuación usando el mandato *'aws login'*:



- 3) Creando el directorio `~/.aws` y copiando las credenciales usando, por ejemplo: `nano ~/.aws/credentials`. El fichero debería tener un aspecto similar al siguiente:

```
[default]
aws_access_key_id = ASIA.....JY3
aws_secret_access_key = p...t/ra.....Qgol

aws_session_token                                     =
IQoJb3JpZ2LuX2VjEPPr////////wEaCXV.....SIbpUPhdovfxKCAiEA5mnbv1L70d5ny8icVPue
sS7oj9h26Xh+9HKfYr/xqwsqtQIIw////////.....
jgoD649EbdVaMGw9+e5HCzX4puK/SNq7F0F
```

Para tener en cuenta también, que la región [10] por defecto se suele encontrar en `~/.aws/config` o puede cambiarse mediante el mandato: *aws configure set region REGION_NUEVA*

En lo relativo a las formas de realizar el despliegue, se dan dos situaciones bastante distinguidas. La primera de ellas consiste en desplegar mediante lo que se conoce como un 'bastión de despliegue'. Esto consiste en utilizar una máquina virtual de la propia nube como punto de entrada del despliegue, lo que elimina la necesidad de instalar dependencias y herramientas en el propio terminal local del usuario mientras que la segunda implica la ejecución local en el ordenador de todo el proceso. Al ser este el primer despliegue del documento, se detallan a continuación ambas formas.

Dando comienzo por el despliegue mediante bastión, el primer paso será lanzar una instancia EC2 que puede tener la configuración por defecto. Ahora se vuelven a abrir varias posibilidades ya que para subir los archivos del despliegue se puede realizar desde el terminal local mediante el mandato `scp` que tendrá una forma similar a la siguiente:

```
scp -i /ruta/clave.pem -r . ec2-user@IP_0_DNS_PUBLICA_EC2:
```

Nótese que la clave puede estar en formato `.pem` o `.ppk` según las preferencias de cada usuario y `'-r .'` copia recursivamente todo el directorio actual y sus subdirectorios por lo que sería necesario estar en la carpeta raíz del despliegue para que todo funcione adecuadamente, de igual manera, se puede sustituir por

una ruta específica como se hace con la clave de acceso a la instancia. Otra forma de obtener los archivos necesarios es mediante GitHub y su mandato 'git clone' que permiten traer los ficheros de un determinado repositorio con el siguiente comando aplicado a este caso: `git clone https://github.com/etsiinf-marcoscr/arch1\_deployment.git` (puede requerir previamente `sudo dnf install git -y`).

Una vez llegados a este punto, convergen ambos métodos de bastión y local. Es necesaria ahora la instalación de las dependencias mediante el mandato 'dnf' en el caso de las EC2 Amazon Linux 2023 o 'apt' en el caso de Ubuntu, entre otras muchas formas existentes de instalar Python, Docker y boto3 (API de AWS para Python). Se muestra ahora la secuencia de comandos necesarios para la máquina EC2:

```
sudo dnf install -y python3-pip docker
pip3 install boto3
```

Llegados a este punto puede comenzarse el proceso de despliegue descrito en el siguiente apartado.

1.2 Proceso de despliegue

Comenzando por la primera arquitectura hay que destacar que el etiquetado de los recursos del despliegue se realiza de manera manual. Este etiquetado es relevante para la rápida localización de los recursos, especialmente cuando se tienen una gran cantidad de recursos desplegados en una misma cuenta AWS. Además, como se verá posteriormente el etiquetado (tagging) permite la monitorización aislada en la aplicación de un subconjunto de los recursos. Esta técnica de etiquetar los servicios desplegados sigue las recomendaciones y buenas prácticas de AWS además de ser una técnica muy usada a nivel más técnico y profesional.

```
TAGS      = [{"Key": "Stack", "Value": "arch1"}]
TAGS_ECS  = [{"key": "Stack", "value": "arch1"}]
TAG_SPECS = lambda resource_type: [{"ResourceType": resource_type, "Tags": TAGS}]
```

En lo que se refiere al procedimiento del despliegue, primero se crean tres repositorios ECR para almacenar las imágenes Docker tanto del Postgres, API Flask y Swagger. La única configuración realizada en este servicio es dar un nombre descriptivo a cada repositorio y el etiquetado con el par clave-valor que se aplicará de aquí en adelante al resto de instancias (Stack-arch1).

```
for repo_name in ["gamestore-postgres", "gamestore-api", "gamestore-swagger"]:
    try:
        ecr.create_repository(
            repositoryName=repo_name,
            tags=TAGS
        )
        print(f"Repositorio creado: {repo_name}")
    except ecr.exceptions.RepositoryAlreadyExistsException:
        print(f"Repositorio ya existe: {repo_name}")
```

Una vez creados los repositorios ECR (cuya creación es casi instantánea), se ejecuta un comando de login para poder subir las imágenes Docker que se van a ir construyendo:

```
def ecr_login():
    print("Login ECR")
    subprocess.run(
        f"aws ecr get-login-password --region {REGION} "
        f"| docker login --username AWS --password-stdin {ECR_ROOT}",
        shell=True, check=True
    )
```

El siguiente paso, es construir y subir las imágenes de postgres:15 (obtenido directamente de Docker Hub) al repositorio gamestore-postgres.

```
def build_push_postgres():
    repo = f"{ECR_ROOT}/gamestore-postgres:latest"
    print("Build/push postgres")
    subprocess.run("docker pull postgres:15", shell=True, check=True)
    subprocess.run(f"docker tag postgres:15 {repo}", shell=True, check=True)
    subprocess.run(f"docker push {repo}", shell=True, check=True)
```

A continuación, se crea el segmento de red cloud (VPC) con el bloque de direcciones (CIDR) 11.0.0.0/16, añadiéndole la etiqueta del despliegue y activando las opciones de resolución de DNS:

```
print("\nCreando VPC")
vpc_id = ec2.create_vpc(
    CidrBlock="11.0.0.0/16",
    TagSpecifications=TAG_SPECS("vpc")
)["Vpc"]["VpcId"]
ec2.modify_vpc_attribute(VpcId=vpc_id, EnableDnsHostnames={"Value": True})
ec2.modify_vpc_attribute(VpcId=vpc_id, EnableDnsSupport={"Value": True})
print("VPC:", vpc_id)
```

Siguiendo con la configuración de red, se crean tres subredes, dos públicas y una privada, en las zonas de disponibilidad us-east-1a y us-east-1b dentro de la región de Virginia del Norte (us-east-1) como se comentó en el apartado de diseño. Se eligió esta región ya que es la que se habilita desde AWS para el curso de Cloud Developing y, en general, es la habilitada para la mayor parte de cursos formativos dadas sus prestaciones y capacidad.

```
azs = ec2.describe_availability_zones(
    Filters=[{"Name": "state", "Values": ["available"]}])["AvailabilityZones"]

az1 = azs[0]["ZoneName"]
az2 = azs[1]["ZoneName"]

public_subnet = ec2.create_subnet(
    VpcId=vpc_id, CidrBlock="11.0.1.0/24", AvailabilityZone=az1,
    TagSpecifications=TAG_SPECS("subnet")
)["Subnet"]["SubnetId"]

public_subnet2 = ec2.create_subnet(
    VpcId=vpc_id, CidrBlock="11.0.3.0/24", AvailabilityZone=az2,
    TagSpecifications=TAG_SPECS("subnet")
)["Subnet"]["SubnetId"]
```

```
private_subnet = ec2.create_subnet(
    VpcId=vpc_id, CidrBlock="11.0.2.0/24", AvailabilityZone=az1,
    TagSpecifications=TAG_SPECS("subnet")
)["Subnet"]["SubnetId"]
```

Como se puede observar el bloque de direcciones de cada subred admite 256 IPs (x.x.x.x/24), quedando la configuración general de red de la siguiente forma:

Nombre subred	Bloque CIDR	Zona de disponibilidad
public_subnet	11.0.1.0/24	us-east-1a (az1)
public_subnet2	11.0.3.0/24	us-east-1a (az1)
private_subnet	11.0.2.0/24	us-east-1b (az2)

Aún con esto, es necesario establecer las tablas de enrutamiento pública y privada de la red junto con la creación de una pasarela a Internet (internet gateway) que permita a las subredes públicas establecer comunicación con el exterior (Internet).

```
print("Creando IGW")
igw = ec2.create_internet_gateway(
    TagSpecifications=TAG_SPECS("internet-gateway")
)["InternetGateway"]["InternetGatewayId"]
ec2.attach_internet_gateway(InternetGatewayId=igw, VpcId=vpc_id)

print("Routing público")
rt_public = ec2.create_route_table(
    VpcId=vpc_id,
    TagSpecifications=TAG_SPECS("route-table")
)["RouteTable"]["RouteTableId"]
ec2.create_route(RouteTableId=rt_public, DestinationCidrBlock="0.0.0.0/0",
GatewayId=igw)
ec2.associate_route_table(SubnetId=public_subnet, RouteTableId=rt_public)
ec2.associate_route_table(SubnetId=public_subnet2, RouteTableId=rt_public)

print("Routing privado")
rt_private = ec2.create_route_table(
    VpcId=vpc_id,
    TagSpecifications=TAG_SPECS("route-table")
)["RouteTable"]["RouteTableId"]
ec2.associate_route_table(SubnetId=private_subnet, RouteTableId=rt_private)
```

Este bloque de código deja configuradas de la siguiente forma las tablas de ruta:
Tabla de enrutamiento pública (rt_public):

Destino	Objetivo
0.0.0.0/0	Internet Gateway (igw-XXXXXX)
11.0.0.0/16	local

Tabla de enrutamiento privada (rt_private):

Destino	Objetivo
VPC Endpoints (Lista de prefijos)	VPC
11.0.0.0/16	local

Ahora se proceden a crear los grupos de seguridad (security groups) encargados de hacer de firewall y proteger los recursos. Para este despliegue se crearán cuatro: uno para los VPC endpoints, otro para el balanceador de carga, otro más que proteja el Swagger y un último encargo de aislar el backend (Postgres y API Flask)

```
def make_sg(name, desc):
    return ec2.create_security_group(
        GroupName=name,
        Description=desc,
        VpcId=vpc_id,
        TagSpecifications=TAG_SPECS("security-group")
    )["GroupId"]

swagger_sg = make_sg("swagger-sg", "swagger")
backend_sg = make_sg("backend-sg", "backend")
alb_sg     = make_sg("alb-sg", "alb")
endpoint_sg = make_sg("endpoint-sg", "endpoint")
```

Acto seguido hay que configurar las reglas de entrada y salida de estos grupos de seguridad. Comenzando por el Swagger, este deberá tener habilitado el puerto 8080 con todas las direcciones IP de tal forma que cualquier usuario pueda acceder a este servicio.

```
ec2.authorize_security_group_ingress(GroupId=swagger_sg, IpPermissions=[{
    "IpProtocol": "tcp", "FromPort": 8080, "ToPort": 8080,
    "IpRanges": [{"CidrIp": "0.0.0.0/0"}]
}])
```

En cuanto al balanceador de carga, este permitirá las conexiones por HTTP desde cualquier dirección IP haciendo posible que cualquier persona pueda interactuar directamente con la API Flask de la subred privada incluido también el servicio Swagger. Esto podría evitarse haciendo solo posible las conexiones desde el grupo de seguridad del servicio Swagger, pero con el fin de que se puedan realizar verificaciones directamente contra la API se ha optado por dejar las peticiones a la API abiertas.

```
ec2.authorize_security_group_ingress(GroupId=alb_sg, IpPermissions=[{
    "IpProtocol": "tcp", "FromPort": 80, "ToPort": 80,
    "IpRanges": [{"CidrIp": "0.0.0.0/0"}]
}])
```

Por otra parte, a los recursos de la subred privada solo se podrá acceder desde el balanceador de carga y con destino el puerto 5000 que corresponde con el del API Flask que atiende las peticiones mencionadas anteriormente. La base de

datos PostgreSQL (puerto 5432) solo será accesible por este propio grupo de seguridad, es decir, que solo podrá interactuar con ella el API Flask.

```
ec2.authorize_security_group_ingress(GroupId=backend_sg, IpPermissions=[
    {"IpProtocol": "tcp", "FromPort": 5000, "ToPort": 5000,
     "UserIdGroupPairs": [{"GroupId": alb_sg}]},
    {"IpProtocol": "tcp", "FromPort": 5432, "ToPort": 5432,
     "UserIdGroupPairs": [{"GroupId": backend_sg}]}
])
```

Y para acabar con los grupos de seguridad es necesario mencionar el grupo de los VPC endpoints que permiten la comunicación controlada por HTTPS (puerto 443) con algunos servicios de AWS, sobre estos se darán más detalles a continuación.

```
ec2.authorize_security_group_ingress(GroupId=endpoint_sg, IpPermissions=[{
    "IpProtocol": "tcp", "FromPort": 443, "ToPort": 443,
    "UserIdGroupPairs": [
        {"GroupId": backend_sg},
        {"GroupId": swagger_sg}
    ]
}])
```

Siguiendo con estos VPC endpoints, estos son necesarios entre otras cosas para poder realizar una depuración detallada de los servicios desplegados mediante logs de Cloudwatch que se suben a cubos (buckets) de S3 para su almacenaje y para que los servicios que se encuentran en la zona privada de la red puedan comunicarse con los repositorios ECR y así puedan obtener las imágenes Docker correspondientes de cada servicio para su posterior puesta en marcha. Sin estos endpoints hubiese hecho falta una pasarela NAT cuyo coste es más elevado.

```
def create_tagged_endpoint(**kwargs):
    ep_id = ec2.create_vpc_endpoint(**kwargs)["VpcEndpoint"]["VpcEndpointId"]
    ec2.create_tags(Resources=[ep_id], Tags=TAGS)
    return ep_id

create_tagged_endpoint(
    VpcId=vpc_id,
    ServiceName=f"com.amazonaws.{REGION}.s3",
    VpcEndpointType="Gateway",
    RouteTableIds=[rt_private]
)

create_tagged_endpoint(
    VpcId=vpc_id,
    ServiceName=f"com.amazonaws.{REGION}.ecr.api",
    VpcEndpointType="Interface",
    SubnetIds=[private_subnet],
    SecurityGroupIds=[endpoint_sg],
    PrivateDnsEnabled=True
)

create_tagged_endpoint(
    VpcId=vpc_id,
```

```

        ServiceName=f"com.amazonaws.{REGION}.ecr.dkr",
        VpcEndpointType="Interface",
        SubnetIds=[private_subnet],
        SecurityGroupIds=[endpoint_sg],
        PrivateDnsEnabled=True
    )

    create_tagged_endpoint(
        VpcId=vpc_id,
        ServiceName=f"com.amazonaws.{REGION}.logs",
        VpcEndpointType="Interface",
        SubnetIds=[private_subnet],
        SecurityGroupIds=[endpoint_sg],
        PrivateDnsEnabled=True
    )

```

El siguiente paso es preparar los permisos para que los contenedores puedan ejecutarse sin problemas. Aquí, de nuevo, al tratarse de un entorno educativo en el que AWS está limitado, no se pueden crear roles y permisos IAM personalizados ni tan siquiera modificar los existentes, es por eso que no se puede cumplir estrictamente el principio de mínimo privilegio que recomienda el propio Amazon, que consiste en garantizar los mínimos permisos a cada recurso y tipo de usuario implicado en el despliegue. La única opción posible, es asignar a los recursos el rol del laboratorio (LabRole), que no cuenta con los máximos permisos de administración (root), pero sí con algunos permisos prescindibles para este caso.

```

iam = boto3.client("iam")
execution_role_arn = iam.get_role(RoleName="LabRole")["Role"]["Arn"]
print("Usando execution role:", execution_role_arn)

```

En ese fragmento de código solo se obtiene el ARN (identificador) del rol para su posterior aplicación en cada servicio.

Ahora se creará el cluster ECS que compondrá los servicios y tareas necesarios para hacer que todo funcione:

```

print("Creando cluster ECS")
ecs.create_cluster(
    clusterName="gamestore-cluster",
    tags=TAGS_ECS
)

```

A partir de aquí, se inicia la construcción de lo que será cada servicio, esto es, una definición de tarea (task definition) y la propia creación del servicio.

Comenzando por la definición de tarea, esta configura la VPC, permisos, capacidad de cómputo y los recursos del contenedor (imagen Docker, mapeado de los puertos y variables de entorno entre otras). Se puede observar ahora la creación de la definición de tarea del Postgres:

```

print("Registrando task postgres")
ecs.register_task_definition(
    family="postgres",
    executionRoleArn=execution_role_arn,

```

```

networkMode="awsvpc",
requiresCompatibilities=["FARGATE"],
cpu="256", memory="512",
tags=TAGS_ECS,
containerDefinitions=[{
    "name": "postgres",
    "image": f"{ECR_ROOT}/gamestore-postgres:latest",
    "portMappings": [{"containerPort": 5432}],
    "environment": [
        {"name": "POSTGRES_DB", "value": "gamestore"},
        {"name": "POSTGRES_USER", "value": "gamestore"},
        {"name": "POSTGRES_PASSWORD", "value": "gamestorepass"}
    ]
}]
)

```

En este caso, se eligen 0,256 vCPU (un cuarto de núcleo de CPU) y 512MB de memoria, que son suficientes para este caso y permiten una mejor gestión del crédito AWS disponible. El puerto que mapear será el de Postgres (5432) y se establecen unas variables de entorno para nombrar la base de datos, un usuario y la contraseña de este, todo esto junto la imagen del contenedor que hace referencia a la última subida en el repositorio ECR de postgres.

En cuanto a la creación del servicio, se indica el cluster donde ejecutará, nombre, definición de tarea a ejecutar, número de contenedores en ejecución deseados, tipo de lanzamiento y configuración de red:

```

print("Creando servicio postgres")
ecs.create_service(
    cluster="gamestore-cluster",
    serviceName="postgres",
    taskDefinition="postgres",
    desiredCount=1,
    launchType="FARGATE",
    tags=TAGS_ECS,
    networkConfiguration={"awsvpcConfiguration": {
        "subnets": [private_subnet],
        "securityGroups": [backend_sg],
        "assignPublicIp": "DISABLED"
    }}
)

```

Lo siguiente, es detener la ejecución del script hasta que la tarea del servicio esté en ejecución (running), para así poder obtener la IP privada de la tarea y enlazar la API con la base de datos. Este procedimiento no es el más adecuado que existe, ya que AWS ofrece un servicio de nombrado interno de servicios (namespace) mediante DNS que permitiría hacer la arquitectura más tolerante a fallos enlazando el DNS de la tarea a la base de datos de tal forma que si el Postgres sufriese algún fallo no hiciese falta tener que volver a lanzar todo el cluster, pero, este recurso no está disponible en el laboratorio utilizado.

```

postgres_task = wait_for_task_running("gamestore-cluster", "postgres")
postgres_ip    = get_task_private_ip(postgres_task)
print("IP privada postgres:", postgres_ip)

```

`wait_for_task_running(cluster, service)` realiza una espera activa que consulta cada 10 segundos la lista de tareas del servicio 'service' en el cluster 'cluster' verificando si el estado de la primera tarea es en ejecución y devuelve la tarea cuando esta ya esté activa.

```

def wait_for_task_running(cluster, service):
    print(f"Esperando que el task '{service}' esté RUNNING...")
    while True:
        task_arns = ecs.list_tasks(cluster=cluster,
service_name=service)["taskArns"]
        if task_arns:
            task = ecs.describe_tasks(cluster=cluster,
tasks=task_arns)["tasks"][0]
            status = task["lastStatus"]
            print(f" Estado actual: {status}")
            if status == "RUNNING":
                return task
        time.sleep(10)

```

`get_task_private_ip(task)` obtiene la IP privada de la tarea 'task' buscando el identificador de interfaz de red de la tarea para luego poder sacar la IP de su interfaz de red a partir de ese ID.

```

def get_task_private_ip(task):
    eni_id = next(
        d["value"]
        for d in task["attachments"][0]["details"]
        if d["name"] == "networkInterfaceId"
    )
    eni_desc = ec2.describe_network_interfaces(NetworkInterfaceIds=[eni_id])
    return eni_desc["NetworkInterfaces"][0]["PrivateIpAddress"]

```

Una vez obtenida esa IP, se comienza con el proceso de construcción de la API Flask. Esta vez, la imagen no proviene de Docker Hub y hay que construirla mediante Dockerfile. Antes de eso, se plantea un nuevo problema, que consiste en esperar a que la base de datos PostgreSQL esté activa y lista porque de lo contrario podrían surgir problemas si el contenedor de la API Flask ya se ha levantado y la base de datos no está lista. Para esquivar este problema se crea el archivo 'wait-for-db.sh.template' cuyo contenido se muestra a continuación:

```

#!/bin/sh

echo "Esperando a PostgreSQL..."

until pg_isready -h POSTGRES_IP -p 5432 -U gamestore
do
    echo "Postgres no está listo..."

```

```

    sleep 2
done

echo "Postgres listo!"

flask run --host=0.0.0.0

```

Este shell script comprueba cada 2 segundos que la base de datos PostgreSQL esté lista y configurada mediante el mandato `pg_isready` con los flags del host (-h) apuntando a la IP privada del Postgres, al puerto (-p) 5432 y con el usuario (-U) `gamestore` creado mediante variable de entorno anteriormente. Cuando la base de datos está preparada se inicia la API. Como se puede observar, la IP privada no está definida todavía y habrá que sustituir el texto `'POSTGRES_IP'` por la dirección obtenida en la función `get_task_private_ip` mediante la función `'render_template'`.

```

def render_template(template_path, output_path, old_value, new_value):
    with open(template_path, "r", encoding="utf-8") as f:
        content = f.read()
        content = content.replace(old_value, new_value)
        content = content.replace("\r\n", "\n")
    with open(output_path, "w", encoding="utf-8") as f:
        f.write(content)

```

Esta función sustituye un valor `'old_value'` por otro `'new_value'` en el fichero ubicado en `'template_path'` dejando el fichero resultante en `'output_path'`, todas ellas variables pasadas como parámetros de la función. Si ya existiese el fichero resultante ubicado en `'output_path'`, este sería sobrescrito. Este comportamiento es el deseado para así no tener que borrar todo el rato los ficheros resultantes de este proceso cada vez que se quiera lanzar de nuevo el despliegue. A continuación, se muestra la llamada a la función desde el script Python principal:

```

render_template(
    "api/wait-for-db.sh.template",
    "api/wait-for-db.sh",
    "POSTGRES_IP",
    postgres_ip
)

```

En cuanto al Dockerfile, este obtiene primero las dependencias necesarias mediante el gestor de paquetes de bajo nivel APT de Linux para poder ejecutar los sucesivos mandatos del fichero.

```

# api/Dockerfile
FROM python:slim-trixie

WORKDIR /app

RUN apt-get update && apt-get install -y \
    gcc \

```

```
libpq-dev \  
postgresql-client \  
&& rm -rf /var/lib/apt/lists/*
```

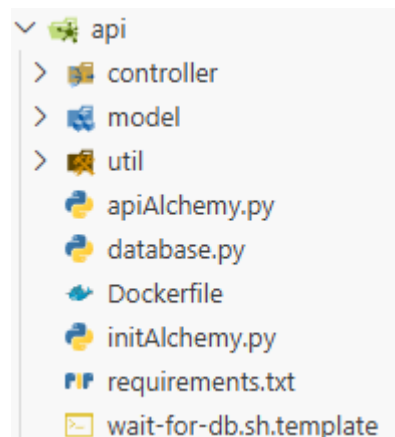
También se instalan las dependencias, ubicadas en el fichero ‘requirements.txt’ mostrado a continuación, relativas a la ejecución de los scripts de Python que componen el API Flask:

```
flask-restful == 0.3.10  
Flask-SQLAlchemy == 3.1.1  
psycopg2-binary == 2.9.10  
flask_cors == 4.0.0  
gunicorn == 21.2.0  
requests == 2.32.5
```

```
# Dependencias Python  
COPY requirements.txt .  
RUN pip install -r requirements.txt
```

Luego se copia todo el contenido del directorio donde está ubicado el Dockerfile

```
COPY . .
```



Lo siguiente es copiar el archivo que se encarga de esperar a que la base de datos esté preparada ‘wait-for-db.sh’. A este archivo se le ejecutará un ‘sed’ para reemplazar posibles caracteres de retorno de carro que resulten incompatibles en UNIX y se le dan posteriormente permisos de ejecución (chmod +x).

```
COPY wait-for-db.sh /wait-for-db.sh  
RUN sed -i 's/\r//' /wait-for-db.sh && chmod +x /wait-for-db.sh
```

Por último, en cuanto al Dockerfile, se establece la variable de entorno que indica el punto de entrada del Flask, se expone el puerto 5000 donde escuchará la API y se ejecuta la espera activa de la base de datos mostrada anteriormente.

```
ENV FLASK_APP=initAlchemy.py  
EXPOSE 5000
```

```
CMD ["/wait-for-db.sh"]
```

Una vez mostrado el proceso de construcción de la imagen Docker se realiza este proceso y la posterior subida al repositorio ECR mediante el script Python:

```
def build_push_api():
    repo = f"{ECR_ROOT}/gamestore-api:latest"
    print("Build/push API")
    subprocess.run("docker build -t gamestore-api ./api", shell=True, check=True)
    subprocess.run(f"docker tag gamestore-api:latest {repo}", shell=True,
check=True)
    subprocess.run(f"docker push {repo}", shell=True, check=True)
# .... otras funciones y codigo .....
build_push_api()
```

Como antes, una vez subida la imagen al repositorio se define la tarea de forma análoga, a excepción, del puerto que es el 5000 y la variable de entorno 'DATABASE_URI' que hace que el Flask apunte a la base de datos.

```
print("Registrando task API")
ecs.register_task_definition(
    family="api",
    executionRoleArn=execution_role_arn,
    networkMode="awsvpc",
    requiresCompatibilities=["FARGATE"],
    cpu="256", memory="512",
    tags=TAGS_ECS,
    containerDefinitions=[{
        "name": "api",
        "image": f"{ECR_ROOT}/gamestore-api:latest",
        "portMappings": [{"containerPort": 5000}],
        "environment": [{
            "name": "DATABASE_URI",
            "value":
f"postgresql://gamestore:gamestorepass@{postgres_ip}:5432/gamestore"
        }]
    }]
)
```

En este momento, se crea el balanceador de carga de la arquitectura para que el futuro servicio Swagger se pueda comunicar con el Flask. Para esto, se le asignan las dos subredes públicas creadas al principio del despliegue, el grupo de seguridad específico para este recurso y se definen otros parámetros de red como IPv4, que sea un balanceador de tipo aplicación (ALB) o que esté disponible de cara a Internet.

```
print("Creando ALB")
alb_resp = elbv2.create_load_balancer(
    Name="api-alb",
    Subnets=[public_subnet, public_subnet2],
    SecurityGroups=[alb_sg],
    Scheme="internet-facing",
    Type="application",
```

```

        IpAddressType="ipv4",
        Tags=TAGS
    )
alb_arn = alb_resp["LoadBalancers"][0]["LoadBalancerArn"]
alb_dns = alb_resp["LoadBalancers"][0]["DNSName"]
print("ALB DNS:", alb_dns)

```

Después se crea el grupo objetivo (target group) del balanceador que indica donde quiere que se balancee la carga. En este caso será únicamente el puerto 5000 correspondiente al Flask y se configura como ruta de comprobación de salud (health check path) el GET /game. Si en algún momento el balanceador no pudiese recibir datos de esa ruta, se marcaría como 'no saludable' (unhealthy) y saldría un aviso en el panel de gestión de AWS.

```

tg_arn = elbv2.create_target_group(
    Name="api-target",
    Protocol="HTTP",
    Port=5000,
    VpcId=vpc_id,
    TargetType="ip",
    HealthCheckPath="/api/game",
    Tags=TAGS
)["TargetGroups"][0]["TargetGroupArn"]

```

La última configuración del balanceador consistirá en crear un punto de entrada (listener) que compruebe las solicitudes de los usuarios mediante el protocolo y puertos especificados y reaccione de una determinada forma. Este listener escuchará peticiones por HTTP en el puerto 80 y las reenviará al grupo objetivo creado antes.

```

elbv2.create_listener(
    LoadBalancerArn=alb_arn,
    Protocol="HTTP", Port=80,
    DefaultActions=[{"Type": "forward", "TargetGroupArn": tg_arn}],
    Tags=TAGS
)

```

Una vez creado el balanceador y sus complementos, se procede a levantar el servicio con la peculiaridad de que se incluye el balanceador como parte de la configuración del servicio.

```

print("Creando servicio API")
ecs.create_service(
    cluster="gamestore-cluster",
    serviceName="api",
    taskDefinition="api",
    desiredCount=1,
    launchType="FARGATE",
    tags=TAGS_ECS,
    loadBalancers=[{
        "targetGroupArn": tg_arn,
        "containerName": "api",
        "containerPort": 5000
    }],
)

```



```

networkConfiguration={"awsvpcConfiguration": {
    "subnets": [private_subnet],
    "securityGroups": [backend_sg],
    "assignPublicIp": "DISABLED"
}}
)

```

Y se fuerza una actualización del servicio para asegurar que los cambios de red en relación con el balanceador de carga ya están propagados y así la tarea ya tiene la configuración correcta. Esto es una medida de garantizar que todo funciona pese a que en la gran mayoría de casos esta actualización resulte innecesaria.

```

ecs.update_service(
    cluster="gamestore-cluster",
    service="api",
    forceNewDeployment=True
)

```

En cuanto al último servicio, el Swagger, se sigue el mismo procedimiento que en el Flask. Primero se construye la imagen, se sube a ECR, se crea la definición de la tarea y se crea el servicio. La particularidad ahora es que hay que cambiar la URL dentro del fichero 'swagger-config.json' para que apunte al balanceador de carga. Para ello, se reutiliza la función 'render_template' que cambiaba una cadena por otra en un fichero y lo almacenaba en una ruta específica. Se muestra ahora la parte a cambiar en el swagger-config.json:

```

{
    "url": "ALB_URL",
    "description": "API en subred privada"
}

```

Y la llamada a la función para realizar el reemplazo:

```

render_template(
    "swagger/swagger-config.json.template",
    "swagger/swagger-config.json",
    "ALB_URL",
    f"http://{alb_dns}/api"
)

```

Una vez hecho esto, se construye la imagen Docker a partir del correspondiente Dockerfile donde se obtienen las dependencias de Swagger, se copia el archivo de configuración resultante del reemplazo realizado y se establece la variable de entorno que permite hacer funcionar el servicio:

```

FROM swaggerapi/swagger-ui:latest

COPY swagger-config.json /swagger.json

ENV SWAGGER_JSON=/swagger.json

```

Llegados a la fase en que el Dockerfile está listo, se procede a la construcción y subida de la imagen a su respectivo repositorio ECR.

```
def build_push_swagger():
    repo = f"{ECR_ROOT}/gamestore-swagger:latest"
    print("Build/push Swagger")
    subprocess.run("docker build -t gamestore-swagger ./swagger", shell=True,
check=True)
    subprocess.run(f"docker tag gamestore-swagger:latest {repo}", shell=True,
check=True)
    subprocess.run(f"docker push {repo}", shell=True, check=True)
# ..... otras funciones y codigo .....
build_push_swagger()
```

Es necesario ahora registrar la definición de la tarea que ejecutará el servicio, esta vez, sin necesidad de variables de entorno y con el puerto 8080.

```
print("Registrando task Swagger")
ecs.register_task_definition(
    family="swagger",
    executionRoleArn=execution_role_arn,
    networkMode="awsvpc",
    requiresCompatibilities=["FARGATE"],
    cpu="256", memory="512",
    tags=TAGS_ECS,
    containerDefinitions=[{
        "name": "swagger",
        "image": f"{ECR_ROOT}/gamestore-swagger:latest",
        "portMappings": [{"containerPort": 8080}]
    }]
)
```

Por último, se crea el servicio Swagger de forma análoga a los dos anteriores a excepción de que al ser un servicio público hay que asignarle una IP pública consecuentemente. De nuevo, se vuelve a forzar el despliegue de la tarea para asegurar que todo se despliega adecuadamente.

```
print("Creando servicio Swagger")
ecs.create_service(
    cluster="gamestore-cluster",
    serviceName="swagger",
    taskDefinition="swagger",
    desiredCount=1,
    launchType="FARGATE",
    tags=TAGS_ECS,
    networkConfiguration={"awsvpcConfiguration": {
        "subnets": [public_subnet],
        "securityGroups": [swagger_sg],
        "assignPublicIp": "ENABLED"
    }}
)

ecs.update_service(
    cluster="gamestore-cluster",
    service="swagger",
    forceNewDeployment=True
)
```

Explicado ya el proceso de despliegue, se indica ahora el comando en Amazon Linux 2023 para ejecutar el script que se acaba de detallar:

```
python3 arch1-deployment.py
```

1.3 Borrado de recursos

Sirva esta sección para documentar como borrar los recursos desplegados en cada una de las arquitecturas anteriormente mencionadas. Siendo en ambos casos el dual a la operación de despliegue.

En la primera arquitectura, el borrado se produce de igual forma que el despliegue, mediante un script escrito en Python. Hay que destacar que el script es idempotente por lo que si en algún momento fallase el borrado de un recurso se podría volver a ejecutar el programa sin ningún inconveniente, ya que los recursos borrados previamente son saltados al ejecutar de nuevo.

Primeramente, se empieza borrando los repositorios ECR, buscando por su nombre y forzando el borrado.

```
title("1. Repositorios ECR")

for repo_name in ["gamestore-postgres", "gamestore-api", "gamestore-swagger"]:
    try:
        ecr.delete_repository(repositoryName=repo_name, force=True)
        ok(f"Repositorio ECR {repo_name} eliminado")
    except ecr.exceptions.RepositoryNotFoundException:
        skip(f"Repositorio ECR {repo_name}")
```

Se continúa borrando ahora ECS, comenzando primero por los servicios. Primero se obtienen sus ARNs para posteriormente escalarlos a 0, es decir, eliminar las tareas en ejecución y luego se obtiene su nombre para borrarlos definitivamente.

```
title("2. Servicios ECS")

try:
    services = ecs.list_services(cluster=CLUSTER)["serviceArns"]
    if services:
        for svc_arn in services:
            svc_name = svc_arn.split("/")[-1]
            ecs.update_service(cluster=CLUSTER, service=svc_name, desiredCount=0)
            ok(f"Servicio {svc_name} escalado a 0")

            print(" Esperando que los tasks se detengan...")
            time.sleep(30)

            for svc_arn in services:
                svc_name = svc_arn.split("/")[-1]
                ecs.delete_service(cluster=CLUSTER, service=svc_name, force=True)
                ok(f"Servicio {svc_name} eliminado")
    else:
        skip("Servicios ECS")
except ecs.exceptions.ClusterNotFoundException:
```

```
skip("Cluster ECS (no existe)")
```

Una vez borrados los servicios, se puede eliminar el cluster por completo.

```
title("3. Cluster ECS")

try:
    ecs.delete_cluster(cluster=CLUSTER)
    ok(f"Cluster {CLUSTER} eliminado")
except ecs.exceptions.ClusterNotFoundException:
    skip("Cluster ECS")
```

Siguiendo ya con las definiciones de las tareas, en este caso, se busca en cada familia de tareas activa los ARNs de estas definiciones de tareas y se dan de baja en los registros del servicio ECS.

```
title("4. Task Definitions")

for family in ["postgres", "api", "swagger"]:
    try:
        paginator = ecs.get_paginator("list_task_definitions")
        arns = []
        for page in paginator.paginate(familyPrefix=family, status="ACTIVE"):
            arns.extend(page["taskDefinitionArns"])
        for arn in arns:
            ecs.deregister_task_definition(taskDefinition=arn)
            ok(f"Task definition desregistrada: {arn.split('/')[-1]}")
        if not arns:
            skip(f"Task definitions familia {family}")
    except Exception as e:
        print(f" ! Error desregistrando {family}: {e}")
```

El balanceador de carga es el siguiente recurso que borrar junto con los listeners (puntos de entrada) asociados a él. Primero se buscan los balanceadores etiquetados y se almacenan en una lista para posteriormente borrar primero sus listeners asociados y luego el propio balanceador.

```
title("5. ALB")

albs = elbv2.describe_load_balancers()["LoadBalancers"]
albs_tagged = [
    a for a in albs
    if any(
        t["Key"] == TAG_KEY and t["Value"] == TAG_VALUE
        for t in elbv2.describe_tags(ResourceArns=[a["LoadBalancerArn"]])
        ["TagDescriptions"][0]["Tags"]
    )
]

for alb in albs_tagged:
    alb_arn = alb["LoadBalancerArn"]

    listeners = elbv2.describe_listeners(LoadBalancerArn=alb_arn)["Listeners"]
    for l in listeners:
```

```

        elbv2.delete_listener(ListenerArn=l["ListenerArn"])
        ok(f"Listener eliminado")

    elbv2.delete_load_balancer(LoadBalancerArn=alb_arn)
    ok(f"ALB {alb['LoadBalancerName']} eliminado, esperando...")
    waiter = elbv2.get_waiter("load_balancers_deleted")
    waiter.wait(LoadBalancerArns=[alb_arn])
    ok("ALB eliminado completamente")

if not albs_tagged:
    skip("ALB")

```

Pero no hay que olvidar tampoco los target groups (grupos objetivo) que no se borran junto con el balanceador y hay que ocuparse de ellos individualmente. La estrategia seguida es obtener la lista de target groups activos y filtrarlos luego por nombre o por etiqueta para finalmente eliminarlos, este proceso es crítico ya que no se pueden eliminar mientras el balanceador esté activo por lo que pueden requerir de un poco más de tiempo para borrarse que el resto de los recursos.

```

print("\n Buscando target groups (asociados y huérfanos)...")

all_tgs = elbv2.describe_target_groups()["TargetGroups"]

tgs_to_delete = []
for tg in all_tgs:
    tg_arn = tg["TargetGroupArn"]
    # Por nombre conocido
    if tg["TargetGroupName"] in ("api-target",):
        tgs_to_delete.append(tg_arn)
        continue
    # Por tag
    try:
        tags =
elbv2.describe_tags(ResourceArns=[tg_arn])["TagDescriptions"][0]["Tags"]
        if any(t["Key"] == TAG_KEY and t["Value"] == TAG_VALUE for t in tags):
            tgs_to_delete.append(tg_arn)
    except Exception:
        pass

for tg_arn in tgs_to_delete:
    for intento in range(6):
        try:
            elbv2.delete_target_group(TargetGroupArn=tg_arn)
            ok(f"Target group eliminado: {tg_arn.split('/')[2]}")
            break
        except elbv2.exceptions.ResourceInUseException:
            print(f"    En uso, esperando 5s... (intento {intento+1}/6)")
            time.sleep(5)
        except Exception as e:
            print(f"    Error: {e}")
            break
    else:
        print(f"    ! No se pudo borrar el target group tras varios intentos")

```

```
if not tgs_to_delete:
    skip("Target groups")
```

Los VPC endpoints son los siguientes en la lista por borrar, al igual que con otros recursos, se filtran los IDs de estos por la etiqueta del despliegue y que no hayan sido ya borrados y se eliminan. Luego se hace una espera activa hasta que estos se eliminan por completo ya que pueden tardar hasta un par de minutos en borrarse y con ellos activos no se puede seguir con el borrado.

```
title("6. VPC Endpoints")

endpoints = ec2.describe_vpc_endpoints(Filters=TAG_FILTER)["VpcEndpoints"]
# Filtrar solo los que no esten ya eliminados
ep_ids = [
    e["VpcEndpointId"] for e in endpoints
    if e["State"] not in ("deleted", "deleting")
]

if ep_ids:
    ec2.delete_vpc_endpoints(VpcEndpointIds=ep_ids)
    ok(f"Solicitud de borrado enviada para {len(ep_ids)} endpoint(s)")

    print(" Esperando que los endpoints se eliminen...")
    while True:
        try:
            still_active = ec2.describe_vpc_endpoints(
                VpcEndpointIds=ep_ids,
                Filters=[{"Name": "vpc-endpoint-state", "Values": ["deleting",
"pending", "available"]}])["VpcEndpoints"]
            if not still_active:
                break
            print(f" Quedan {len(still_active)} endpoint(s) activos, esperando
10s...")
        except ec2.exceptions.ClientError as e:
            if "InvalidVpcEndpointId.NotFound" in str(e):
                # Todos los endpoints han desaparecido: borrado completado
                break
            raise
        time.sleep(10)
    ok("Todos los endpoints eliminados")
else:
    skip("VPC Endpoints")
```

Continuando con el internet gateway (pasarela a Internet), se obtiene el id de la VPC para liberar cualquier recurso asociado a una dirección pública en esa VPC. Una vez no hay recursos mapeados en la VPC, se obtiene el internet gateway de la red y se desasocia para su eliminación.

```
title("7. Internet Gateway")

sleep(15) # Esperar un poco para que AWS actualice el estado de los recursos
tras eliminar endpoints
```

```

vpcs = ec2.describe_vpcs(Filters=TAG_FILTER)["Vpcs"]
vpc_id = vpcs[0]["VpcId"] if vpcs else None

if vpc_id:
    # Liberar cualquier IP pública/EIP mapeada en la VPC antes de desconectar el
    IGW
    enis = ec2.describe_network_interfaces(
        Filters=[{"Name": "vpc-id", "Values": [vpc_id]}]
    )["NetworkInterfaces"]

    for eni in enis:
        assoc = eni.get("Association")
        if assoc:
            allocation_id = assoc.get("AllocationId")
            association_id = assoc.get("AssociationId")
            # Si tiene AssociationId es que la IP pública está asociada a esta
            ENI, desasociar primero
            if association_id:
                try:
                    ec2.disassociate_address(AssociationId=association_id)
                    print(f"    IP pública desasociada de ENI
{eni['NetworkInterfaceId']}")
                except Exception as e:
                    print(f"    No se pudo desasociar IP: {e}")
            # Si tiene AllocationId es una EIP, liberar también
            if allocation_id:
                try:
                    ec2.release_address(AllocationId=allocation_id)
                    print(f"    EIP {allocation_id} liberada")
                except Exception as e:
                    print(f"    No se pudo liberar EIP: {e}")

    igws = ec2.describe_internet_gateways(
        Filters=[{"Name": "attachment.vpc-id", "Values": [vpc_id]}]
    )["InternetGateways"]
    for igw in igws:
        igw_id = igw["InternetGatewayId"]
        ec2.detach_internet_gateway(InternetGatewayId=igw_id, VpcId=vpc_id)
        ec2.delete_internet_gateway(InternetGatewayId=igw_id)
        ok(f"IGW {igw_id} desconectado y eliminado")
    if not igws:
        skip("IGW")
else:
    skip("VPC (y por tanto IGW)")

```

Lo siguiente será eliminar las tres subredes creadas siempre y cuando estas no tengan ENIs activos asociados, es decir, que no haya recursos en la subred que se estén usando.

```

title("8. Subnets")

if vpc_id:
    subnets = ec2.describe_subnets(Filters=TAG_FILTER)["Subnets"]

```

```

for subnet in subnets:
    for intento in range(12): # hasta 60 segundos de espera por subnet
        try:
            ec2.delete_subnet(SubnetId=subnet["SubnetId"])
            ok(f"Subnet {subnet['SubnetId']} eliminada")
            break
        except ec2.exceptions.ClientError as e:
            if "DependencyViolation" in str(e):
                # Buscar ENIs huérfanos en esta subnet y eliminarlos
                enis = ec2.describe_network_interfaces(
                    Filters=[{"Name": "subnet-id", "Values":
[subnet["SubnetId"]]}]
                )["NetworkInterfaces"]
                if enis:
                    for eni in enis:
                        eni_id = eni["NetworkInterfaceId"]
                        status = eni["Status"]
                        print(f"    ENI huérfano detectado: {eni_id} (estado:
{status})")

                        # Solo se pueden borrar ENIs en estado 'available'
                        if status == "available":
                            try:
                                ec2.delete_network_interface(NetworkInterface
Id=eni_id)

                                print(f"    ENI {eni_id} eliminado")
                            except Exception as e2:
                                print(f"    No se pudo borrar ENI: {e2}")
                            else:
                                print(f"    ENI {eni_id} no está disponible aún,
esperando...")
                        else:
                            print(f"    Sin ENIs visibles, esperando liberación...
(intento {intento+1}/12)")
                            time.sleep(5)
                    else:
                        raise
            else:
                print(f" ! No se pudo borrar subnet {subnet['SubnetId']} tras varios
intentos")
            if not subnets:
                skip("Subnets")
        else:
            skip("Subnets (sin VPC)")

```

El próximo paso incluirá borrar las tablas de ruta una vez las subredes que tengan asociadas se hayan borrado excluyendo la tabla de ruta principal de la cuenta AWS.

```
title("9. Route Tables")
```



```

if vpc_id:
    rts = ec2.describe_route_tables(Filters=TAG_FILTER)["RouteTables"]
    for rt in rts:
        is_main = any(a.get("Main") for a in rt.get("Associations", []))
        if is_main:
            skip(f"Route table main {rt['RouteTableId']}")
            continue
        for assoc in rt.get("Associations", []):
            if not assoc.get("Main"):
                ec2.disassociate_route_table(AssociationId=assoc["RouteTableAssociationId"])
        ec2.delete_route_table(RouteTableId=rt["RouteTableId"])
        ok(f"Route table {rt['RouteTableId']} eliminada")
    if not rts:
        skip("Route Tables")
else:
    skip("Route Tables (sin VPC)")

```

Por último, antes de poder eliminar la VPC al completo, habrá que suprimir los grupos de seguridad que actúan como cortafuegos (firewalls) de la arquitectura. Esto se realiza quitando las reglas de entrada de cada grupo de seguridad, dado que solo ha hecho falta añadir reglas de entrada y no de salida y luego eliminando definitivamente cada grupo.

```

title("10. Security Groups")

if vpc_id:
    sgs = ec2.describe_security_groups(Filters=TAG_FILTER)["SecurityGroups"]
    sgs = [sg for sg in sgs if sg["GroupName"] != "default"]

    for sg in sgs:
        if sg["IpPermissions"]:
            try:
                ec2.revoke_security_group_ingress(
                    GroupId=sg["GroupId"],
                    IpPermissions=sg["IpPermissions"]
                )
            except Exception:
                pass

    for sg in sgs:
        try:
            ec2.delete_security_group(GroupId=sg["GroupId"])
            ok(f"SG {sg['GroupName']} ({sg['GroupId']}) eliminado")
        except Exception as e:
            print(f" ! No se pudo borrar {sg['GroupName']}: {e}")
    if not sgs:
        skip("Security Groups")
else:
    skip("Security Groups (sin VPC)")

```

Finalizando con el borrado de la arquitectura, se elimina el último recurso ahora que se ha liberado completamente de asociaciones.

```

title("11. VPC")

if vpc_id:
    ec2.delete_vpc(VpcId=vpc_id)
    ok(f"VPC {vpc_id} eliminada")
else:
    skip("VPC")

```

Debe recordarse, que en caso de haber realizado el despliegue mediante un bastión EC2, este último recurso deberá borrarse de manera independiente al resto.

1.4 Costes aproximados

Estimate summary [Info](#)

Upfront cost

0.00 USD

Monthly cost

43.64 USD

Total 12 months cost

523.68 USD

Includes upfront cost

Getting Started with AWS

Get started for free

Contact Sales

My Estimate

Q Find resources

Service Name

Upfront cost

Monthly cost

Description

Region

Config Summary

AWS Fargate

0.00 USD

3.94 USD

-

US East (N. Virginia)

Operating system (Linux), CPU Archi...

Elastic Load Balancing

0.00 USD

17.60 USD

-

US East (N. Virginia)

Number of Application Load Balance...

Amazon Virtual Private Cloud (VPC)

0.00 USD

0.00 USD

-

US East (N. Virginia)

Working days per month (22), Numb...

Amazon CloudWatch

0.00 USD

0.00 USD

-

US East (N. Virginia)

Number of mobile OTEL events and ...

Amazon Virtual Private Cloud (VPC)

0.00 USD

22.10 USD

-

US East (N. Virginia)

Working days per month (22), Numb...

<https://calculator.aws/#/estimate?id=4202adc4db4775588c78a72594727e432c323067>